

Seed Data in Entity Framework Core (EF Core)

In this article, I will discuss **Seed Data in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [Transactions in Entity Framework Core \(EF Core\)](#) with Examples.

What Is Seed Data in Entity Framework Core?

In Entity Framework Core (EF Core), seed data refers to the process of prepopulating a database with initial or default data during the database creation or migration process. This is useful for setting up default data that an application requires to function correctly, such as predefined roles, administrative accounts, or master data like countries, states, and cities.

So, Data Seeding allows us to automatically populate the database with certain records as soon as the application is deployed or after migrations run. Seed Data helps developers test their applications with predefined data.

Real-Time Example: Seeding Country, State, and City Master Data

Let's see an example of how to seed data into database tables using Entity Framework Core. Let's create an example of seeding master data for Country, State, and City tables using both EF Core migrations. So, let us proceed and see how to Implement this using Entity Framework core Seed Data.

Defining Entity Models

First, we need to define the Country, State, and City entities as follows:

Country Entity

Create a class file named **Country.cs** within the Entities and then copy and paste the following code. This will create the Country master table and have one-to-many relationships with the State table. That is, one country can have multiple states.

```
using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    [Table("Countries")]
    public class Country
    {
        [Key]
        public int CountryId { get; set; }
        [Required]
        [MaxLength(100)]
        public string CountryName { get; set; }
        [Required]
        [MaxLength(10)]
        public string CountryCode { get; set; }
        // Navigation Property
        public ICollection<State> States { get; set; }
    }
}
```

State Entity

Create a class file named **State.cs** within the Entities and then copy and paste the following code. This will create the States master table and have one-to-many relationships with the Cities table. That is, one State can have multiple Cities.

```
using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    [Table("States")]
    public class State
    {
        [Key]
        public int StateId { get; set; }
        [Required]
        [MaxLength(100)]
        public string StateName { get; set; }
        public int CountryId { get; set; } // Foreign Key
        // Navigation Properties
        public Country Country { get; set; }
        public ICollection<City> Cities { get; set; }
    }
}
```

City Entity

Create a class file named **City.cs** within the Entities and then copy and paste the following code. This will create the City master table and have a one-to-one relationship with the States table. That one city belongs to a single State.

```
using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    [Table("Cities")]
    public class City
    {
        [Key]
        public int CityId { get; set; }
        [Required]
        [MaxLength(100)]
        public string CityName { get; set; }
        public int StateId { get; set; } // Foreign Key
        // Navigation Property
        public State State { get; set; }
    }
}
```

Seeding Data with the HasData Method:

EF Core provides a fluent API method called **HasData** that we can use inside our **OnModelCreating** method of the **DbContext** class. EF Core will create or update the necessary records in our database when we run migrations. The HasData method accepts the entities we want to seed. The following are the steps:

- Override OnModelCreating: In your DbContext class, override the OnModelCreating method.
- Use HasData: Call **modelBuilder.Entity<TEntity>().HasData(...)** to specify the seed data.

Configuring DbContext with Seed Data

Now, let us see how to seed the data using EF Core Migrations. To seed data using EF Core migrations, we need to **Override** the **OnModelCreating** method of the **DbContext** class and use the **HasData** method to specify the initial data for **Each Entity**. The HasData method seeds the country, state, and city master data when the database is created. So, modify the EFCoreDbContext class as follows:

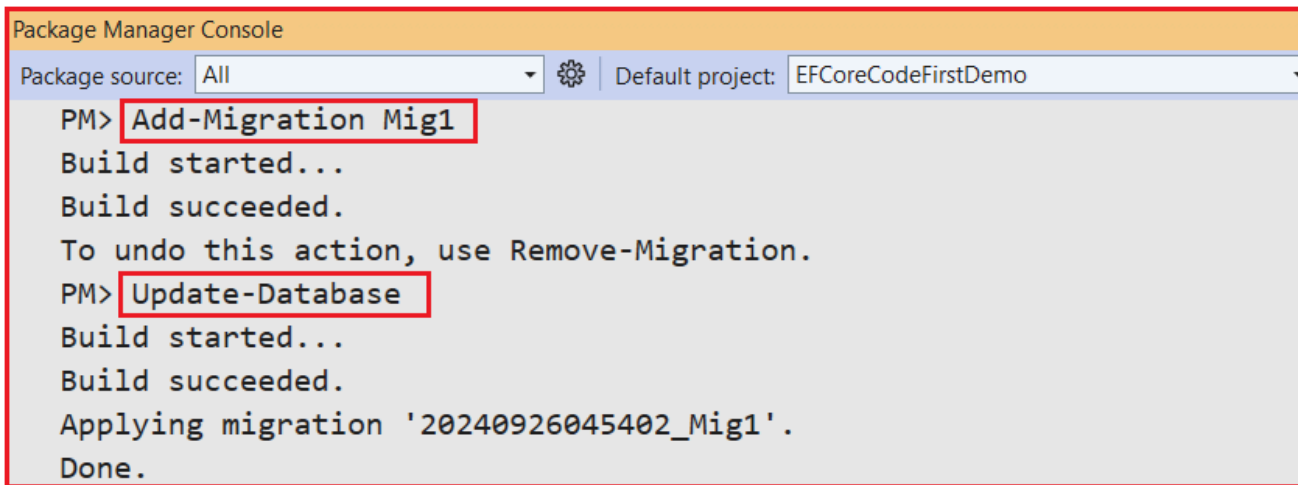
```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the database connection
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Seed Countries Master Data
            modelBuilder.Entity<Country>().HasData(
                new Country { CountryId = 1, CountryName = "India", CountryCode = "IN" },
                new Country { CountryId = 2, CountryName = "Australia", CountryCode = "AU" }
            );
            // Seed States Master Data
            modelBuilder.Entity<State>().HasData(
                new State { StateId = 1, StateName = "Odisha", CountryId = 1 },
                new State { StateId = 2, StateName = "Delhi", CountryId = 1 },
                new State { StateId = 3, StateName = "New South Wales", CountryId = 2 }
            );
            // Seed Cities Master Data
            modelBuilder.Entity<City>().HasData(
                new City { CityId = 1, CityName = "Bhubaneswar", StateId = 1 },
                new City { CityId = 2, CityName = "Cuttack", StateId = 1 },
                new City { CityId = 3, CityName = "New Delhi", StateId = 2 },
                new City { CityId = 4, CityName = "Sydney", StateId = 3 }
            );
        }
        // DbSet representing the tables
        public DbSet<Country> Countries { get; set; }
        public DbSet<State> States { get; set; }
        public DbSet<City> Cities { get; set; }
    }
}
```

}

Note: While using the HasData method, we need to provide the Identity column values manually. When seeding data involving relationships (e.g., foreign keys), ensure that related entities are seeded correctly and that foreign key values are properly set.

Applying Migrations and Updating the Database

After configuring models and seed data, we need to create and apply migrations to update the database schema and insert the seed data. So, open the Package Manager Console and then execute the **Add-Migration and Update-Database** commands as shown in the below image:



The screenshot shows the Package Manager Console window for the project 'EFCoreCodeFirstDemo'. The 'Package source' is set to 'All'. The console output shows the following commands and results:

```
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240926045402_Mig1'.
Done.
```

Verifying Seed Data in Program Class:

To confirm that the seed data has been successfully inserted into the database, we can query the database and display the results. For a better understanding, please modify the Program class as follows:

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())
                {
                    // Retrieve and display Countries
                    Console.WriteLine("=== Country Master Data ===");
                    var countries = context.Countries.ToList();
                    foreach (var country in countries)
                    {
                        Console.WriteLine($"Country ID: {country.CountryId},
Name: {country.CountryName}, Code: {country.CountryCode}");
                    }
                }
            }
        }
    }
}
```

```

// Retrieve and display States
Console.WriteLine("\n=== State Master Data ===");
var states = context.States
    .Include(s => s.Country)
    .ToList();
foreach (var state in states)
{
    Console.WriteLine($"State ID: {state.StateId}, Name:
{state.StateName}, Country: {state.Country.CountryName}");
}
// Retrieve and display Cities
Console.WriteLine("\n=== City Master Data ===");
var cities = context.Cities
    .Include(c => c.State)
    .ThenInclude(s => s.Country)
    .ToList();
foreach (var city in cities)
{
    Console.WriteLine($"City ID: {city.CityId}, Name:
{city.CityName}, State: {city.State.StateName}, Country:
{city.State.Country.CountryName}");
}
}
}
catch (Exception ex)
{
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

Output:

```

=== Country Master Data ===
Country ID: 1, Name: India, Code: IND
Country ID: 2, Name: Australia, Code: AUS

=== State Master Data ===
State ID: 1, Name: Odisha, Country: India
State ID: 2, Name: Delhi, Country: India
State ID: 3, Name: New South Wales, Country: Australia

=== City Master Data ===
City ID: 1, Name: Bhubaneswar, State: Odisha, Country: India
City ID: 2, Name: Cuttack, State: Odisha, Country: India
City ID: 3, Name: New Delhi, State: Delhi, Country: India
City ID: 4, Name: Sydney, State: New South Wales, Country: Australia

```

When to Use Seed Data in Entity Framework Core

Seeding is highly useful during development when we need test data without manually inserting records every time the database is refreshed:

- **Master Data Initialization:** Populating tables with static data like countries, currencies, or categories.
- **Default User Roles and Permissions:** Setting up roles like Admin, User, and their associated permissions.
- **Configuration Settings:** Inserting default configuration settings required for the application.

Seeding data in Entity Framework Core is a powerful feature that facilitates the initialization of databases with default or necessary data. The `HasData` method provides an easy way to do this in the `OnModelCreating` method. This is mainly used for development and testing purposes. For production environments, you typically wouldn't want to have seed data that overwrites or conflicts with existing data.

Custom Initialization for Seed Data in EF Core

While the `HasData` method is convenient for small and simple seed data, it might not be suitable for larger datasets or more complex initialization logic. In such cases, we can implement custom initialization logic. This involves manually ensuring the database is created and inserting seed data if it doesn't already exist. The following are the Steps we need to follow for Custom Initialization for Seed Data:

- **Define a Seed Data Method:** Create a static class with a method to seed data.
- **Call the Seed Method:** Invoke this method during application startup.
- **Handle Transactions and Errors:** Use transactions to ensure data integrity.

Create a Static Initializer Class

Create a static class file named **DbInitializer** within the Entities folder, and then copy and paste the following code. Within this static class, we have defined one static method called `Initialize`. The following class code is self-explained, so please read the comment lines for a better understanding.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public static class DbInitializer
    {
        public static void Initialize(EFCoreDbContext context)
        {
            // Ensure the database is created
            context.Database.EnsureCreated();
            // Check if the database has been seeded
            if (context.Countries.Any() || context.States.Any() ||
context.Cities.Any())
            {
                Console.WriteLine("Database already seeded.");
                return;
            }
            using var transaction = context.Database.BeginTransaction();
            try
            {
                // Seed Countries
                var countries = new List<Country>
                {
                    new Country { CountryName = "India", CountryCode = "IND"

```


Modify the DbContext Class

Since we are handling data seeding in the DbInitializer, we can remove the HasData method from the OnModelCreating method. So, modify the EFCoreDbContext class as follows:

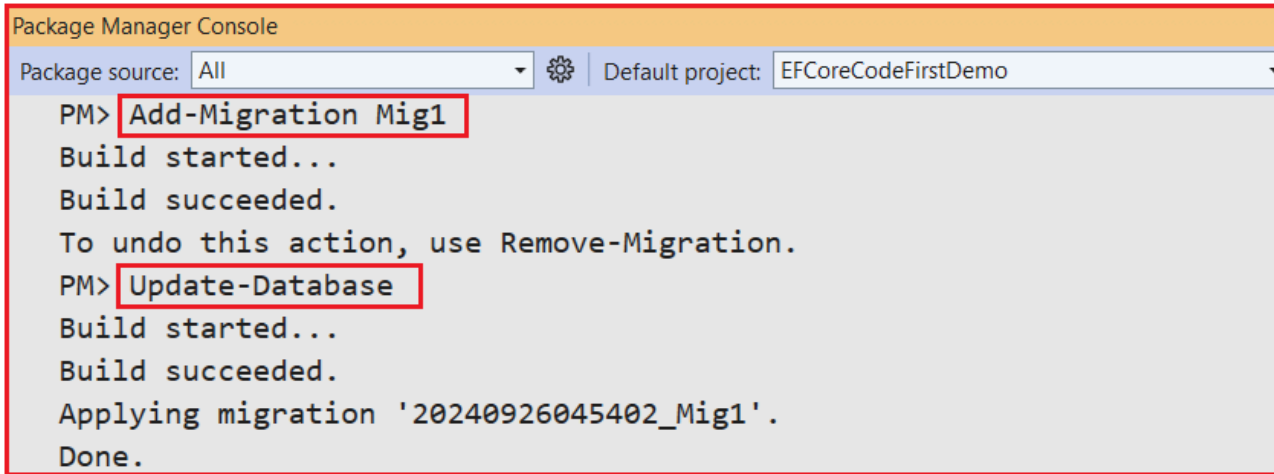
```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the database connection
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Configure the Country entity
            modelBuilder.Entity<Country>(entity =>
            {
                // Set the primary key
                entity.HasKey(c => c.CountryId);
                // Configure the one-to-many relationship between Country and State
                entity.HasMany(c => c.States) // A Country has many States
                    .WithOne(s => s.Country) // Each State has one Country
                    .HasForeignKey(s => s.CountryId) // Foreign key in State
                    .OnDelete(DeleteBehavior.Cascade); // Enable Cascade Delete
            });
            // Configure the State entity
            modelBuilder.Entity<State>(entity =>
            {
                // Set the primary key
                entity.HasKey(s => s.StateId);
                // Configure the one-to-many relationship between State and City
                entity.HasMany(s => s.Cities) // A State has many Cities
                    .WithOne(c => c.State) // Each City has one State
                    .HasForeignKey(c => c.StateId) // Foreign key in City to State
                    .OnDelete(DeleteBehavior.Cascade); // Enable Cascade Delete
            });
            // DbSet representing the tables
            public DbSet<Country> Countries { get; set; }
            public DbSet<State> States { get; set; }
            public DbSet<City> Cities { get; set; }
        }
    }
}
```

Recreate the Database

Since we have changed the way we seed data, it's a good idea to recreate the database:

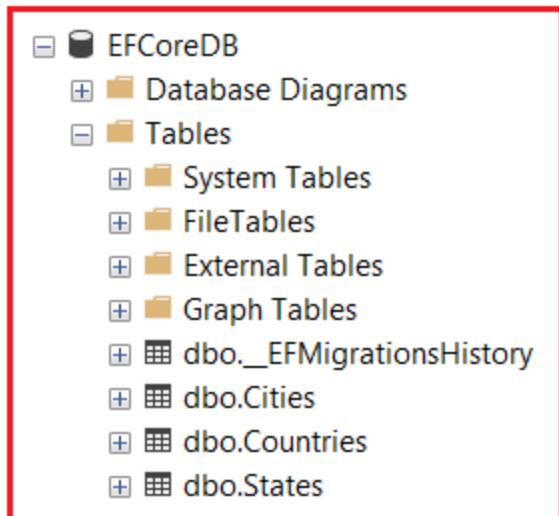
- Delete the existing database. Please make sure to delete the EFCoreDB database
- Remove any existing migrations. It is better to delete the Migration folder from your project.

Then, create a new migration and update the database using the Package Manager Console as follows:



```
Package Manager Console
Package source: All [v] [gear] Default project: EFCoreCodeFirstDemo
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240926045402_Mig1'.
Done.
```

Now, once you execute the above command, it should create the EFCoreDB database with the required database tables, as shown in the below image:



At this point, if you verify the database table, you will see no data. This is because we have not yet called the Database_INITIALIZER method when we start the application.

Call the Seed Method from the Program Class

Modify the Program Class as follows to Invoke the_INITIALIZER, which will seed the data.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {

```

```

public static void Main(string[] args)
{
    try
    {
        using (var context = new EFCoreDbContext())
        {
            // Initialize and seed the database
            DbInitializer.Initialize(context);
            // Display the seeded data
            // Retrieve and display Countries
            Console.WriteLine("=== Country Master Data ===");
            var countries = context.Countries.ToList();
            foreach (var country in countries)
            {
                Console.WriteLine($"Country ID: {country.CountryId},
Name: {country.CountryName}, Code: {country.CountryCode}");
            }
            // Retrieve and display States
            Console.WriteLine("\n=== State Master Data ===");
            var states = context.States
                .Include(s => s.Country)
                .ToList();
            foreach (var state in states)
            {
                Console.WriteLine($"State ID: {state.StateId}, Name:
{state.StateName}, Country: {state.Country.CountryName}");
            }
            // Retrieve and display Cities
            Console.WriteLine("\n=== City Master Data ===");
            var cities = context.Cities
                .Include(c => c.State)
                .ThenInclude(s => s.Country)
                .ToList();
            foreach (var city in cities)
            {
                Console.WriteLine($"City ID: {city.CityId}, Name:
{city.CityName}, State: {city.State.StateName}, Country:
{city.State.Country.CountryName}");
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"An error occurred: {ex.Message}");
    }
}
}

```

Output:

```
Database has been seeded successfully.
=== Country Master Data ===
Country ID: 1, Name: India, Code: IND
Country ID: 2, Name: Australia, Code: AUS

=== State Master Data ===
State ID: 1, Name: Odisha, Country: India
State ID: 2, Name: Delhi, Country: India
State ID: 3, Name: New South Wales, Country: Australia

=== City Master Data ===
City ID: 1, Name: Bhubaneswar, State: Odisha, Country: India
City ID: 2, Name: Cuttack, State: Odisha, Country: India
City ID: 3, Name: New Delhi, State: Delhi, Country: India
City ID: 4, Name: Sydney, State: New South Wales, Country: Australia
```

In the next article, I will discuss [Shadow Properties in Entity Framework Core](#) with Examples. In this article, I explain **Seed Data in Entity Framework Core (EF Core)** with Examples. I hope you enjoy this Seed Data in EF Core article.